

Computer Architecture

CS 622 / EE 520

Fall 2026

Lecture 9
Shahid Masud

- Quick Review of Pipeline and Hazards
- MIPS R4000 Deep Pipeline Example
- Scoreboard in Dynamic Branch Prediction
- Instruction Level Parallelism – Chapter 3
- Parallelism through Loop Unrolling

Quiz 2
Next Lecture

Reading Notes

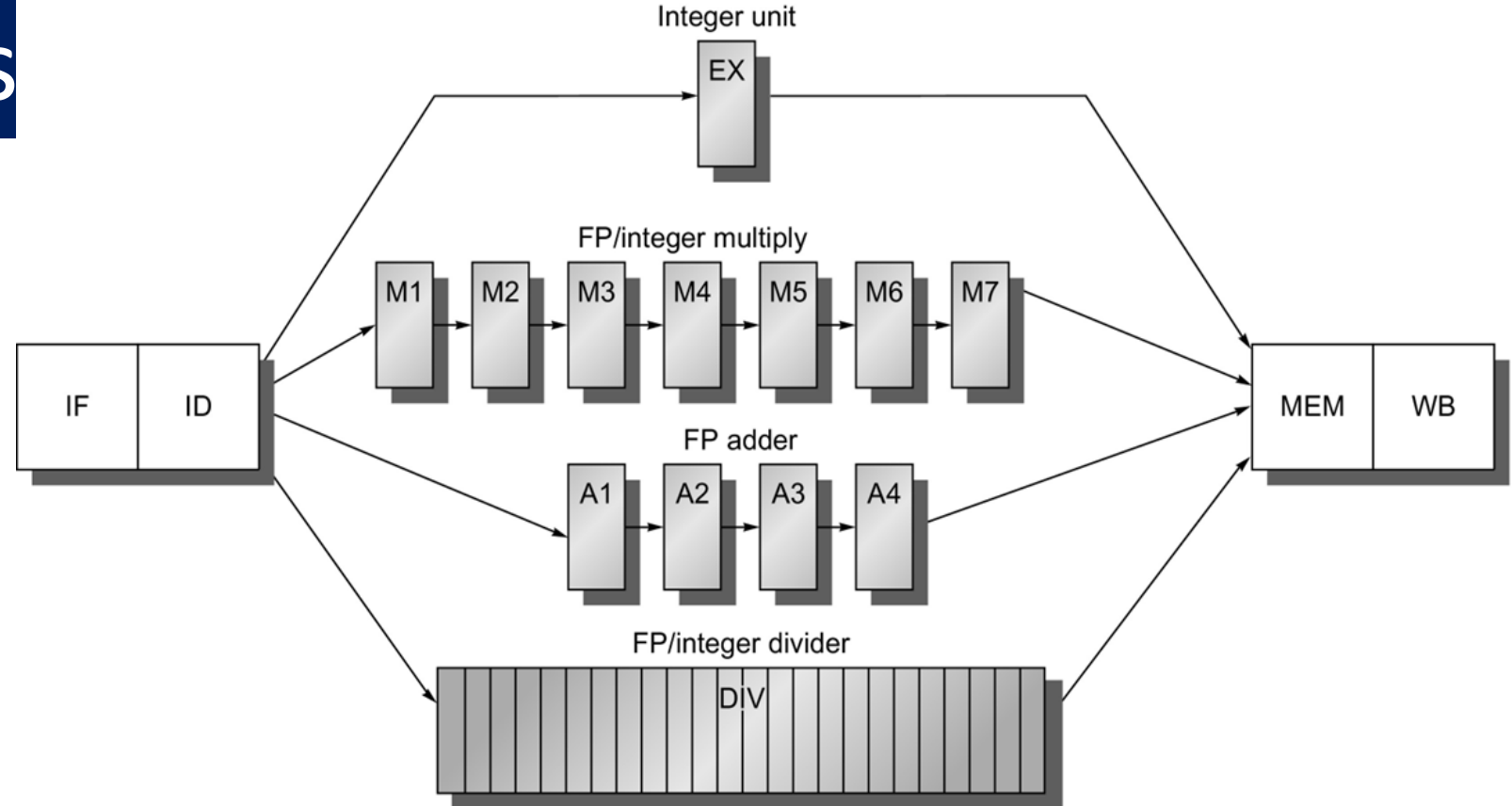
Readings:

Appendix **C of course textbook**

Chapter **3 of course textbook**

Adding Registers

Multicycle Pipeline



The structure of the pipeline in Figure C.30 requires the introduction of the additional pipeline registers (e.g., A1/A2, A2/A3, A3/A4) and the modification of the connections to those registers. The ID/EX register must be expanded to connect ID to EX, DIV, M1, and A1; we can refer to the portion of the register associated with one of the next stages with the notation ID/EX, ID/DIV, ID/M1, or ID/A1. The pipeline register between ID and all the other stages may be thought of as logically separate registers and may, in fact, be implemented as separate registers. Because only one operation can be in a pipe stage at a time, the control information can be associated with the register at the head of the stage.

Problem With Writes in Multicycle Pipeline



Now look at the problems arising from writes, described as (2) and (3) in the earlier list. If we assume that the FP register file has one write port, sequences of FP operations, as well as an FP load together with FP operations, can cause conflicts for the register write port. Consider the pipeline sequence shown in Figure C.33. In clock cycle 11, all three instructions will reach WB and want to write the register file. With only a single register file write port, the processor must serialize the instruction completion. This single register port represents a structural hazard. We could increase the number of write ports to solve this, but that solution may be unattractive because the additional write ports would be used only rarely. This is because the maximum steady-state number of write ports needed is 1. Instead, we choose to detect and enforce access to the write port as a structural hazard.

Implement this Interlock

There are two different ways to implement this interlock. The first is to track the use of the write port in the ID stage and to stall an instruction before it issues, just as we would for any other structural hazard. Tracking the use of the write port can be done with a shift register that indicates when already-issued instructions will use the register file. If the instruction in ID needs to use the register file at the same time as an instruction already issued, the instruction in ID is stalled for a cycle. On each clock the reservation register is shifted 1 bit. This implementation has an advantage: It maintains the property that all interlock detection and stall insertion occurs in the ID stage. The cost is the addition of the shift register and write conflict logic. We will assume this scheme throughout this section.

An alternative scheme is to stall a conflicting instruction when it tries to enter either the MEM or WB stage. If we wait to stall the conflicting instructions until they want to enter the MEM or WB stage, we can choose to stall either instruction. A simple, though sometimes suboptimal, heuristic is to give priority to the unit with the longest latency, because that is the one most likely to have caused another instruction to be stalled for a RAW hazard. The advantage of this scheme is that it does not require us to detect the conflict until the entrance of the MEM or WB stage, where it is easy to see. The disadvantage is that it complicates pipeline control, as stalls can now arise from two places. Notice that stalling before entering MEM will cause the EX, A4, or M7 stage to be occupied, possibly forcing the stall to trickle back in the pipeline. Likewise, stalling before WB would cause MEM to back up.

Our other problem is the possibility of WAW hazards. To see that these exist, consider the example in Figure C.33. If the `fadd.d` instruction were issued one cycle earlier and had a destination of `f2`, then it would create a WAW hazard, because it would write `f2` one cycle earlier than the `fadd.d`. Note that this hazard only occurs when the result of the `fadd.d` is overwritten *without any instruction ever using it*! If there were a use of `f2` between the `fadd.d` and the `fadd.d`, the pipeline would need to be stalled for a RAW hazard, and the `fadd.d` would not issue until the `fadd.d` was completed. We could argue that, for our pipeline, WAW hazards only occur when a useless instruction is executed, but we must still detect them and make sure that the result of the `fadd.d` appears in `f2` when we are done. (As we will see in Section C.8, such sequences sometimes *do* occur in reasonable code.)

Instruction	Clock cycle number									
	1	2	3	4	5	6	7	8	9	10
<code>fmul.d f0, f4, f6</code>	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM
...		IF	ID	EX	MEM	WB				
...			IF	ID	EX	MEM	WB			
<code>fadd.d f2, f4, f6</code>				IF	ID	A1	A2	A3	A4	MEM
...					IF	ID	EX	MEM	WB	
...						IF	ID	EX	MEM	WB
<code>fld f2, 0(x2)</code>							IF	ID	EX	MEM

Figure C.33 Three instructions want to perform a write-back to the FP register file simultaneously, as shown in clock cycle 11. This is *not* the worst case, because an earlier divide in the FP unit could also finish on the same clock. Note that although the `fmul.d`, `fadd.d`, and `fld` are in the MEM stage in clock cycle 10, only the `fld` actually uses the memory, so no structural hazard exists for MEM.

How to handle this WAW Hazard?



There are two possible ways to handle this WAW hazard. The first approach is to delay the issue of the load instruction until the `fadd.d` enters MEM. The second approach is to stamp out the result of the `fadd.d` by detecting the hazard and changing the control so that the `fadd.d` does not write its result. Then the `fadd.d` can issue right away. Because this hazard is rare, either scheme will work fine—you can pick whatever is simpler to implement. In either case, the hazard can be detected during ID when the `fadd.d` is issuing, and stalling the `fadd.d` or making the `fadd.d` a no-op is easy. The difficult situation is to detect that the `fadd.d` might finish before the `fadd.d`, because that requires knowing the length of the pipeline and the current position of the `fadd.d`. Luckily, this code sequence (two writes with no intervening read) will be very rare, so we can use a simple solution: if an instruction in ID wants to write the same register as an instruction already issued, do not issue the instruction to EX. In

①

②

What To check while issuing an Instruction?



sional moves needed between the two register sets.) Assuming that the pipeline does all hazard detection in ID, there are **three checks that must be performed before an instruction can issue:**

1. **Check for structural hazards**—Wait until the required functional unit is not busy (this is only needed for divides in this pipeline) and make sure the register write port is available when it will be needed.
2. **Check for a RAW data hazard**—Wait until the source registers are not listed as pending destinations in a pipeline register that will not be available when this instruction needs the result. A number of checks must be made here, depending on both the source instruction, which determines when the result will be available, and the destination instruction, which determines when the value is needed. For example, if the instruction in ID is an FP operation with source register f2, then f2 cannot be listed as a destination in ID/A1, A1/A2, or A2/A3, which correspond to FP add instructions that will not be finished when the instruction in ID needs a result. (ID/A1 is the portion of the output register of ID that is sent to A1.) Divide is somewhat more tricky, if we want to allow the last few cycles of a divide to be overlapped, because we need to handle the case when a divide is close to finishing as special. In practice, designers might ignore this optimization in favor of a simpler issue test.
3. **Check for a WAW data hazard**—Determine if any instruction in A1, ..., A4, D, M1, ..., M7 has the same register destination as this instruction. If so, stall the issue of the instruction in ID.

Out of Order Completion

Out of Order Completion



Another problem caused by these long-running instructions can be illustrated with the following sequence of code:

```
fdiv.d      f0, f2, f4
fadd.d      f10, f10, f8
fsub.d      f12, f12, f14
```

This code sequence looks straightforward; there are no dependences. A problem arises, however, because an instruction issued early may complete after an instruction issued later. In this example, we can expect `fadd.d` and `fsub.d` to complete before the `fdiv.d` completes. This is called out-of-order completion and is common in pipelines with long-running operations (see [Section C.7](#)). Because hazard



Problem with Out-of-Order Completion



```
fdiv.d    f0,f2,f4
fadd.d    f10,f10,f8
fsub.d    f12,f12,f14
```

why is out-of-order completion a problem? Suppose that the `fsub.d` causes a floating-point arithmetic exception at a point where the `fadd.d` has completed but the `fdiv.d` has not. The result will be an imprecise exception, something we are trying to avoid. It may appear that this could be handled by letting the floating-point pipeline drain, as we do for the integer pipeline. But the exception may be in a position where this is not possible. For example, if the `fdiv.d` decided to take a floating-point-arithmetic exception after the add completed, we could not have a precise exception at the hardware level. In fact, because the `fadd.d` destroys one of its operands, we could not restore the state to what it was before the `fdiv.d`, even with software help.



Solution to out of order completion



1 than they were issued. There are four possible approaches to dealing with out-of-order completion. The first is to ignore the problem and settle for imprecise exceptions. This approach was used in the 1960s and early 1970s. It was still used in some supercomputers in the past fifteen years, where certain classes of exceptions were not allowed or were handled by the hardware without stopping the pipeline. It is difficult to use this approach in most processors built today because of

2 A second approach is to buffer the results of an operation until all the operations that were issued earlier are complete. Some processors actually use this solution, but it becomes expensive when the difference in running times among operations is large, because the number of results to buffer can become large. Furthermore,

3 A third technique in use is to allow the exceptions to become somewhat imprecise, but to keep enough information so that the trap-handling routines can create a precise sequence for the exception. This means knowing what operations were in the pipeline and their PCs. Then, after handling the exception, the software finishes any instructions that precede the latest instruction completed, and the sequence can restart. Consider the following worst-case code sequence:

4 The final technique is a hybrid scheme that allows the instruction issue to continue only if it is certain that all the instructions before the issuing instruction will complete without causing an exception. This guarantees that when an exception occurs, no instructions after the interrupting one will be completed and all of the instructions before the interrupting one can be completed. This sometimes

MIPS R4000 Deep Pipeline Example

branch in the MIPS ISA. The R4000 implements MIPS64 but uses a deeper pipeline than that of our five-stage design both for integer and FP programs. This deeper pipeline allows it to achieve higher clock rates by decomposing the five-stage integer pipeline into eight stages. Because cache access is particularly time critical, the extra pipeline stages come from decomposing the memory access. This type of deeper pipelining is sometimes called *superpipelining*.

pipeline. Notice that, although the instruction and data memory occupy multiple cycles, they are fully pipelined, so that a new instruction can start on every clock.

8 Stage Pipeline of MIPS R4000

- IF: First half of instruction fetch; PC selection actually happens here, together with initiation of instruction cache access.
- IS: Second half of instruction fetch, complete instruction cache access.
- RF: Instruction decode and register fetch, hazard checking, and instruction cache hit detection.
- EX: Execution, which includes effective address calculation, ALU operation, and branch-target computation and condition evaluation.
- DF: Data fetch, first half of data cache access.
- DS: Second half of data fetch, completion of data cache access.
- TC: Tag check, to determine whether the data cache access hit.
- WB: Write-back for loads and register-register operations.

SUPERPIPELINING

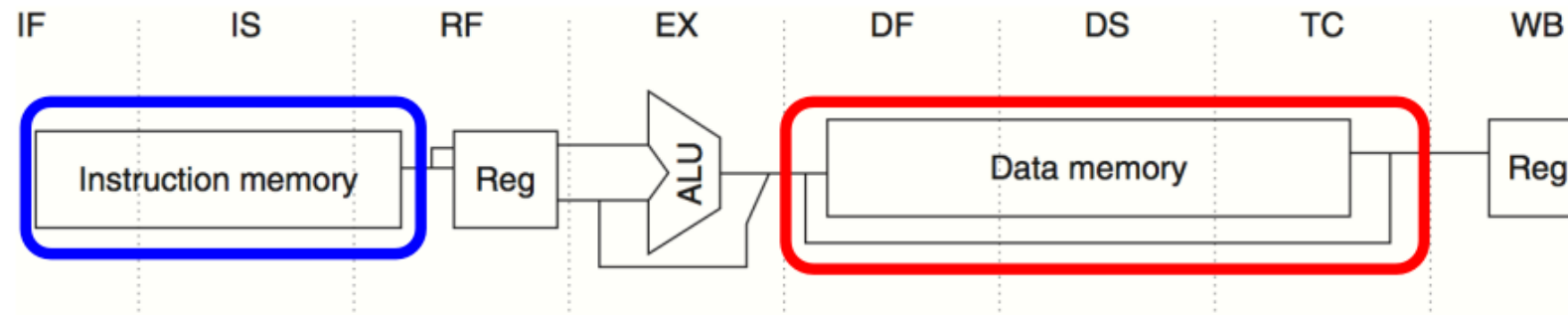
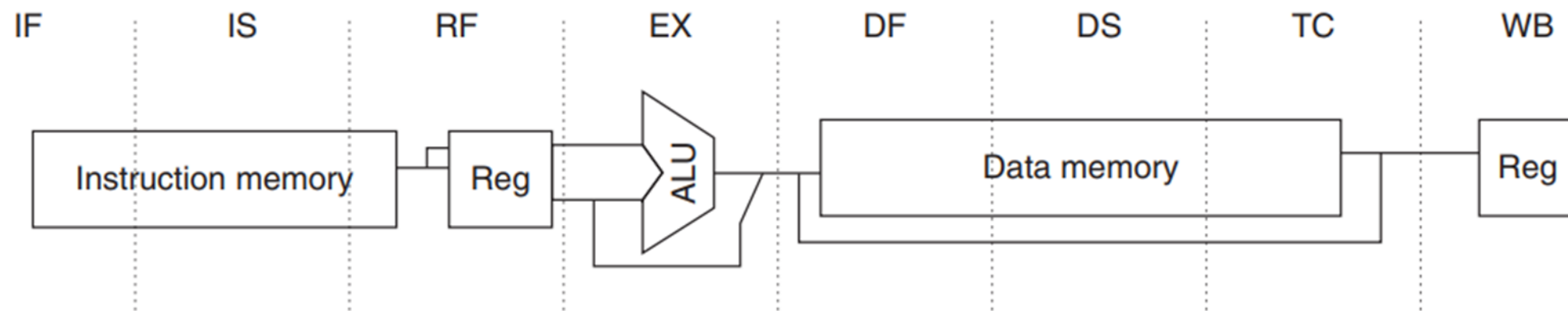


Figure C.36 The eight-stage pipeline structure of the R4000 uses pipelined instruction and data caches. The pipe stages are labeled and their detailed function is described in the text. The vertical dashed lines represent the stage boundaries as well as the location of pipeline latches. The instruction is actually available at the end of IS, but the tag check is done in RF, while the registers are fetched. Thus, we show the instruction memory as operating through RF. The TC stage is needed for data memory access, because we cannot write the data into the register until we know whether the cache access was a hit or not.

Complexity in Architecture

In addition to substantially increasing the amount of forwarding required, this longer-latency pipeline increases both the load and branch delays. Figure C.37 shows that load delays are two cycles, because the data value is available at the end of DS. Figure C.38 shows the shorthand pipeline schedule when a use immediately follows a load. It shows that forwarding is required for the result of a load instruction to a destination that is three or four cycles later.



In addition to substantially increasing the amount of forwarding required, this longer-latency pipeline increases both the load and branch delays. [Figure C.37](#) shows that load delays are two cycles, because the data value is available at the end of DS. [Figure C.38](#) shows the shorthand pipeline schedule when a use immediately follows a load. It shows that forwarding is required for the result of a load instruction to a destination that is three or four cycles later.

2 Cycle Load Delays

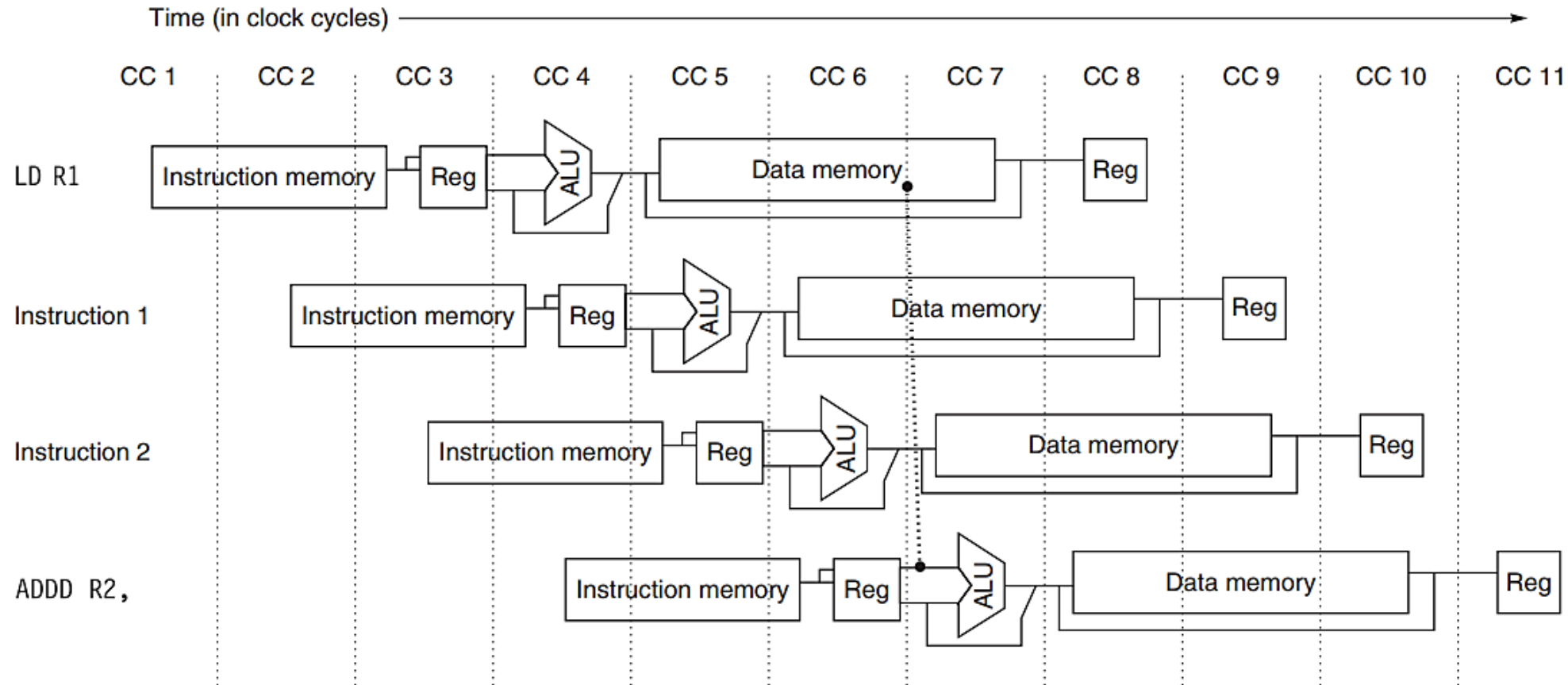


Figure C.37 The structure of the R4000 integer pipeline leads to a x1 load delay. A x1 delay is possible because the data value is available at the end of DS and can be bypassed. If the tag check in TC indicates a miss, the pipeline is backed up a cycle, when the correct data are available.

Figure C.39 shows that the basic branch delay is three cycles, because the branch condition is computed during EX. The MIPS architecture has a single-cycle delayed branch. The R4000 uses a predicted-not-taken strategy for the remaining two cycles of the branch delay. As Figure C.40 shows, untaken branches are simply one-cycle delayed branches, while taken branches have a one-cycle delay slot followed by two idle cycles. The instruction set provides a branch-likely instruction, which we described earlier and which helps in filling the branch delay slot.

Forwarding Example MIPS 4000

Instruction number	Clock number								
	1	2	3	4	5	6	7	8	9
ld x1,...	IF	IS	RF	EX	DF	DS	TC	WB	
add x2,x1,...		IF	IS	RF	Stall	Stall	EX	DF	DS
sub x3,x1,...			IF	IS	Stall	Stall	RF	EX	DF
or x4,x1,...				IF	Stall	Stall	IS	RF	EX

Figure C.38 A load instruction followed by an immediate use results in a x1 stall. Normal forwarding paths can be used after two cycles, so the `add` and `sub` get the value by forwarding after the stall. The `or` instruction gets the value from the register file. Because the two instructions after the load could be independent and hence not stall, the bypass can be to instructions that are three or four cycles after the load.

In addition to the increase in stalls for loads and branches, the deeper pipeline increases the number of levels of forwarding for ALU operations. In our RISC V five-stage pipeline, forwarding between two register-register ALU instructions could happen from the ALU/MEM or the MEM/WB registers. In the R4000 pipeline, there are four possible sources for an ALU bypass: EX/DF, DF/DS, DS/TC, and TC/WB.

Basic Branch Delay

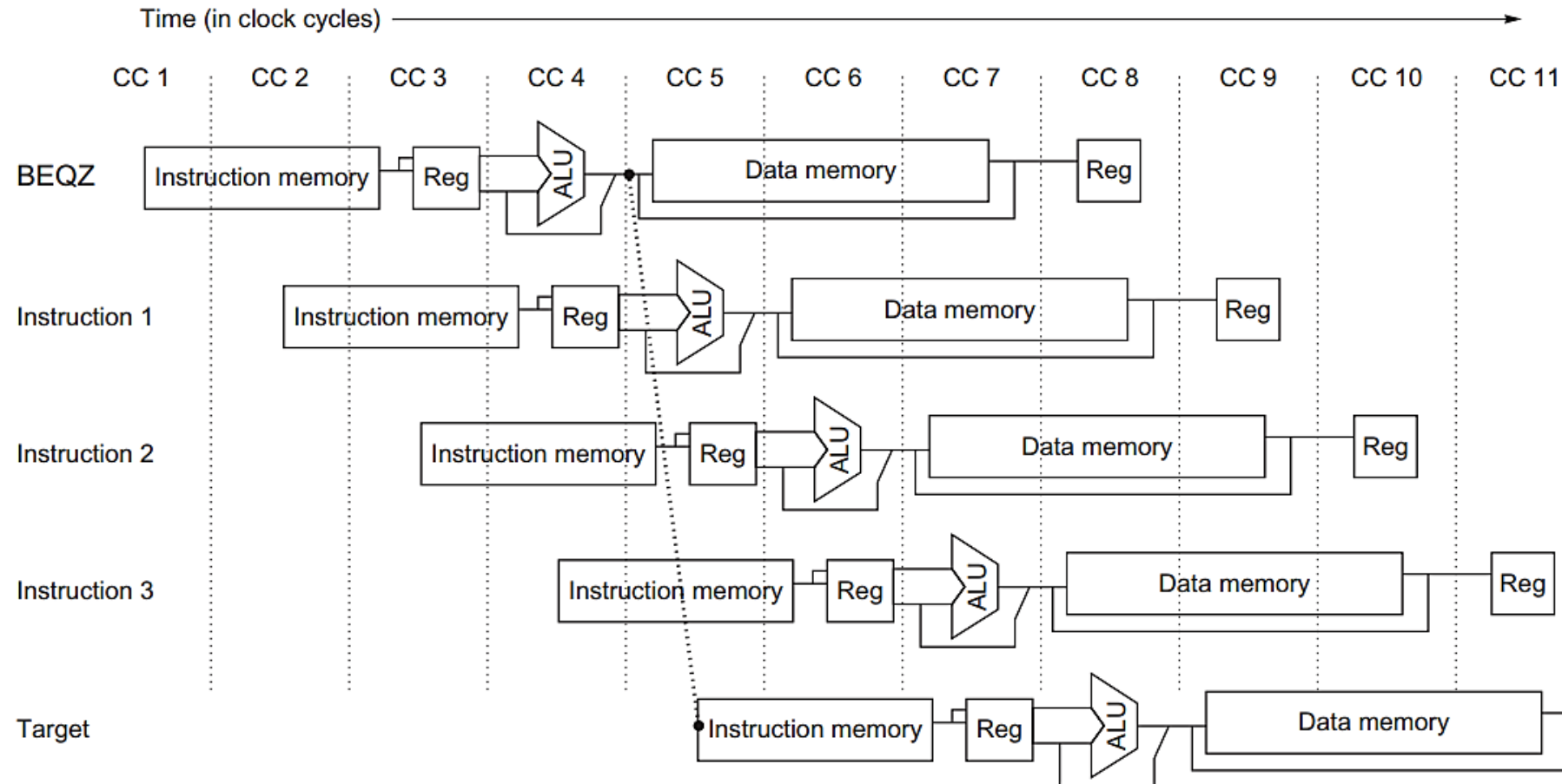


Figure C.39 The basic branch delay is three cycles, because the condition evaluation is performed during EX.

Pipeline interlocks enforce both the x1 branch stall penalty on a taken branch and any data hazard stall that arises from use of a load result. After the R4000, all implementations of MIPS processor made use of dynamic branch prediction.

Hazards and Forwarding in Longer Latency Pipelines

There are a number of different aspects to the hazard detection and forwarding for a pipeline like that shown in [Figure C.30](#).

1. Because the divide unit is not fully pipelined, **structural hazards can occur**. These will need to be detected and issuing instructions will need to be stalled.
2. Because the instructions **have varying running times, the number of register writes required in a cycle can be larger than 1**.
3. **Write after write (WAW) hazards are possible, because instructions no longer reach WB in order**. Note that **write after read (WAR) hazards are not possible, because the register reads always occur in ID**.
4. **Instructions can complete in a different order than they were issued**, causing problems with exceptions; we deal with this in the next subsection.
5. **Because of longer latency of operations, stalls for RAW hazards will be more frequent**.

Figure C.39 shows that the basic branch delay is three cycles, because the branch condition is computed during EX. The MIPS architecture has a single-cycle delayed branch. The R4000 uses a predicted-not-taken strategy for the remaining two cycles of the branch delay. As Figure C.40 shows, untaken branches are simply one-cycle delayed branches, while taken branches have a one-cycle delay slot followed by two idle cycles. The instruction set provides a branch-likely instruction, which we described earlier and which helps in filling the branch delay slot.

Deeper Pipelines and Forwarding



In addition to the increase in stalls for loads and branches, the deeper pipeline increases the number of levels of forwarding for ALU operations. In our RISC V five-stage pipeline, forwarding between two register-register ALU instructions could happen from the ALU/MEM or the MEM/WB registers. In the R4000 pipeline, there are four possible sources for an ALU bypass: EX/DF, DF/DS, DS/TC, and TC/WB.

3 Cycle Branch Delay When Taken (x1 Interlock)

Instruction number	Clock number								
	1	2	3	4	5	6	7	8	9
Branch instruction	IF	IS	RF	EX	DF	DS	TC	WB	
Delay slot		IF	IS	RF	EX	DF	DS	TC	WB
Stall			Stall	Stall	Stall	Stall	Stall	Stall	Stall
Stall				Stall	Stall	Stall	Stall	Stall	Stall
Branch target					IF	IS	RF	EX	DF
Branch instruction	IF	IS	RF	EX	DF	DS	TC	WB	
Delay slot		IF	IS	RF	EX	DF	DS	TC	WB
Branch instruction + 2			IF	IS	RF	EX	DF	DS	TC
Branch instruction + 3				IF	IS	RF	EX	DF	DS

Figure C.40 A taken branch, shown in the top portion of the figure, has a one-cycle delay slot followed by a x1 stall, while an untaken branch, shown in the bottom portion, has simply a one-cycle delay slot. The branch instruction can be an ordinary delayed branch or a branch-likely, which cancels the effect of the instruction in the delay slot if the branch is untaken.

The Floating-Point Pipeline

The R4000 floating-point unit consists of three functional units: a floating-point divider, a floating-point multiplier, and a floating-point adder. The adder logic is used on the final step of a multiply or divide. Double-precision FP operations can take from 2 cycles (for a negate) up to 112 cycles (for a square root). In addition, the various units have different initiation rates. The FP functional unit can be thought of as having eight different stages, listed in [Figure C.41](#); these stages are combined in different orders to execute various FP operations.

There is a single copy of each of these stages, and various instructions may use a stage zero or more times and in different orders. [Figure C.42](#) shows the latency, initiation rate, and pipeline stages used by the most common double-precision FP operations.

FP Stages

MIPS 4000

Stage	Functional unit	Description
A	FP adder	Mantissa add stage
D	FP divider	Divide pipeline stage
E	FP multiplier	Exception test stage
M	FP multiplier	First stage of multiplier
N	FP multiplier	Second stage of multiplier
R	FP adder	Rounding stage
S	FP adder	Operand shift stage
U		Unpack FP numbers

Figure C.41 The eight stages used in the R4000 floating-point pipelines.

FP instruction	Latency	Initiation interval	Pipe stages
Add, subtract	4	3	U, S+A, A+R, R+S
Multiply	8	4	U, E+M, M, M, M, N, N+A, R
Divide	36	35	U, A, R, D ²⁸ , D+A, D+R, D+A, D+R, A, R
Square root	112	111	U, E, (A+R) ¹⁰⁸ , A, R
Negate	2	1	U, S
Absolute value	2	1	U, S
FP compare	3	2	U, A, R

Figure C.42 The latencies and initiation intervals for the FP operations initiation intervals for the FP operations both depend on the FP unit stages that a given operation must use. The latency values assume that the destination instruction is an FP operation; the latencies are one cycle less when the destination is a store. The pipe stages are shown in the order in which they are used for any operation. The notation S+A indicates a clock cycle in which both the S and A stages are used. The notation D²⁸ indicates that the D stage is used 28 times in a row.

		Clock cycle												
Operation	Issue/stall	0	1	2	3	4	5	6	7	8	9	10	11	12
Multiply	Issue	U	E+M	M	M	M	N	N+A	R					
Add	Issue		U	S+A	A+R	R+S								
	Issue			U	S+A	A+R	R+S							
	Issue				U	S+A	A+R	R+S						
	Stall					U	S+A	A+R	R+S					
	Stall						U	S+A	A+R	R+S				
	Issue							U	S+A	A+R	R+S			
	Issue								U	S+A	A+R	R+S		

Figure C.43 An FP multiply issued at clock 0 is followed by a single FP add issued between clocks 1 and 7. The second column indicates whether an instruction of the specified type stalls when it is issued n cycles later, where n is the clock cycle number in which the U stage of the second instruction occurs. The stage or stages that cause a stall are in bold. Note that this table deals with only the interaction between the multiply and *one* add issued between clocks 1 and 7. In this case, the add will stall if it is issued four or five cycles after the multiply; otherwise, it issues without stalling. Notice that the add will be stalled for two cycles if it issues in cycle 4 because on the next clock cycle it will still conflict with the multiply; if, however, the add issues in cycle 5, it will stall for only 1 clock cycle, because that will eliminate the conflicts.

Operation	Issue/stall	Clock cycle												
		0	1	2	3	4	5	6	7	8	9	10	11	12
Add	Issue	U	S+A	A+R	R+S									
Multiply	Issue		U	E+M	M	M	M	N	N+A	R				
	Issue			U	M	M	M	M	N	N+A	R			

Figure C.44 A multiply issuing after an add can always proceed without stalling, because the shorter instruction clears the shared pipeline stages before the longer instruction reaches them.

Operation	Issue/stall	Clock cycle											
		25	26	27	28	29	30	31	32	33	34	35	36
Divide	Issued in cycle 0...	D	D	D	D	D	D+A	D+R	D+A	D+R	A	R	
Add	Issue		U	S+A	A+R	R+S							
	Issue			U	S+A	A+R	R+S						
	Stall				U	S+A	A+R	R+S					
	Stall					U	S+A	A+R	R+S				
	Stall						U	S+A	A+R	R+S			
	Stall							U	S+A	A+R	R+S		
	Stall								U	S+A	A+R	R+S	
	Stall									U	S+A	A+R	R+S
	Issue										U	S+A	A+R
	Issue											U	S+A
	Issue												U

Figure C.45 An FP divide can cause a stall for an add that starts near the end of the divide. The divide starts at cycle 0 and completes at cycle 35; the last 10 cycles of the divide are shown. Because the divide makes heavy use of the rounding hardware needed by the add, it stalls an add that starts in any of cycles 28–33. Notice that the add starting in cycle 28 will be stalled until cycle 36. If the add started right after the divide, it would not conflict, because the add could complete before the divide needed the shared stages, just as we saw in [Figure C.44](#) for a multiply and add. As in the earlier figure, this example assumes *exactly* one add that reaches the U stage between clock cycles 26 and 35.

Operation	Issue/stall	Clock cycle												
		0	1	2	3	4	5	6	7	8	9	10	11	12
Add	Issue	U	S + A	A + R	R + S									
Divide	Stall		U	A	R	D	D	D	D	D	D	D	D	D
	Issue			U	A	R	D	D	D	D	D	D	D	D
	Issue				U	A	R	D	D	D	D	D	D	D

Figure C.46 A double-precision add is followed by a double-precision divide. If the divide starts one cycle after the add, the divide stalls, but after that there is no conflict.

From the data in [Figures C.47](#) and [C.48](#), we can see the penalty of the deeper pipelining. The R4000's pipeline has much longer branch delays than the classic five-stage pipeline. The longer branch delay substantially increases the cycles spent on branches, especially for the integer programs with a higher branch frequency. This is the reason that almost all subsequent processors with moderate to deep pipelines (8–16 stages are typical today) employ dynamic branch predictors.

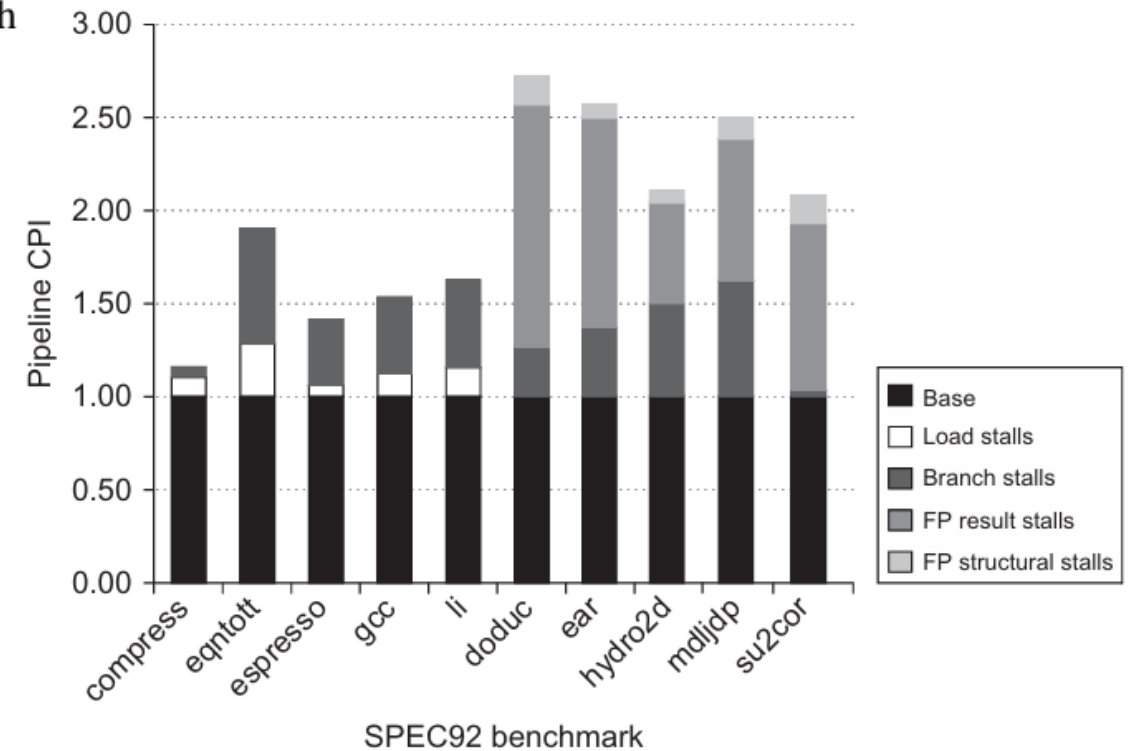


Figure C.47 The pipeline CPI for 10 of the SPEC92 benchmarks, assuming a perfect cache. The pipeline CPI varies from 1.2 to 2.8. The left-most five programs are integer programs, and branch delays are the major CPI contributor for these. The right-most five programs are FP, and FP result stalls are the major contributor for these. [Figure C.48](#) shows the numbers used to construct this plot.

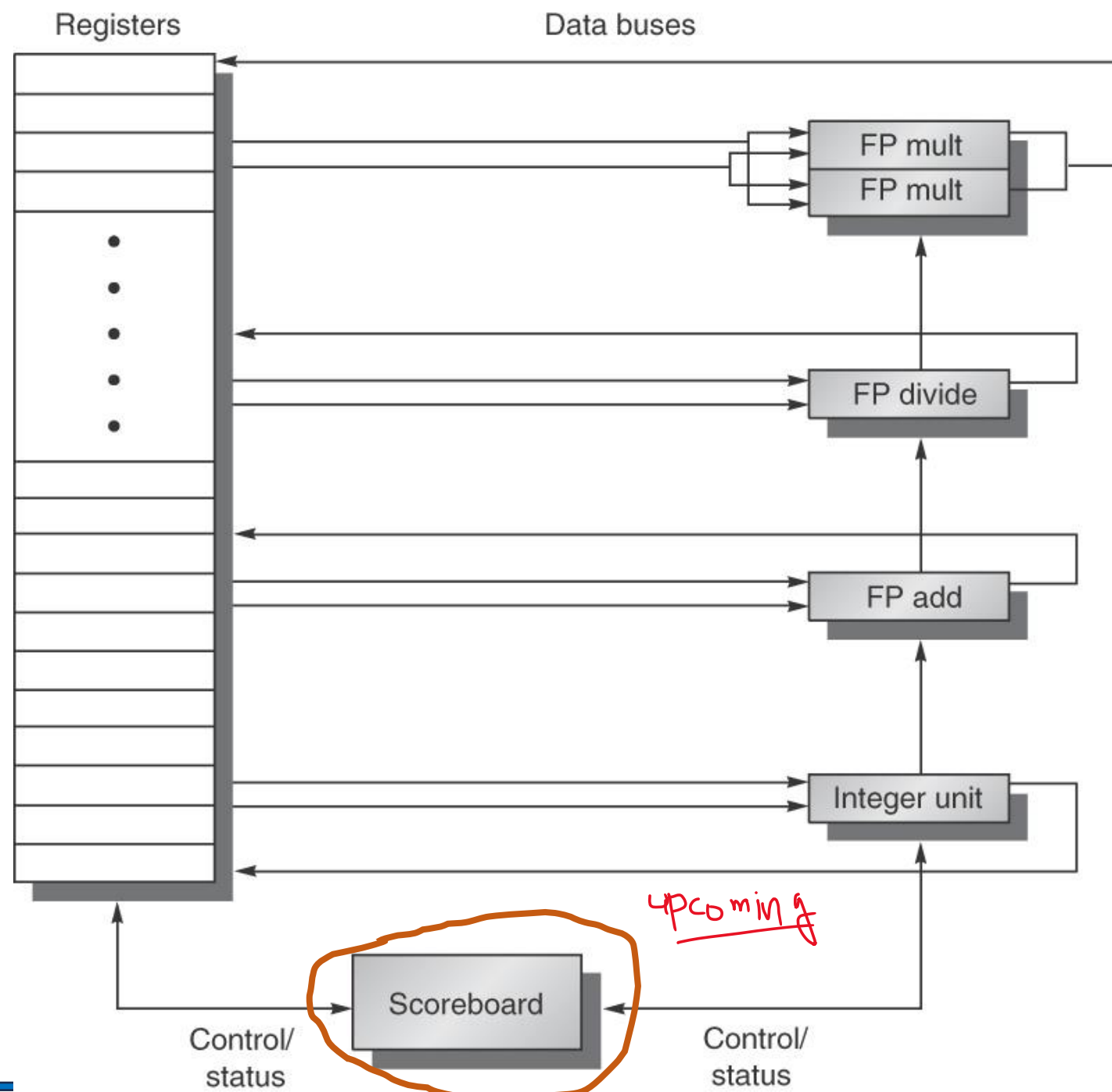
Benchmark	Pipeline CPI	Load stalls	Branch stalls	FP result stalls	FP structural stalls
Compress	1.20	0.14	0.06	0.00	0.00
Eqntott	1.88	0.27	0.61	0.00	0.00
Espresso	1.42	0.07	0.35	0.00	0.00
Gcc	1.56	0.13	0.43	0.00	0.00
Li	1.64	0.18	0.46	0.00	0.00
Integer average	1.54	0.16	0.38	0.00	0.00
Doduc	2.84	0.01	0.22	1.39	0.22
Mdljdp2	2.66	0.01	0.31	1.20	0.15
Ear	2.17	0.00	0.46	0.59	0.12
Hydro2d	2.53	0.00	0.62	0.75	0.17
Su2cor	2.18	0.02	0.07	0.84	0.26
FP average	2.48	0.01	0.33	0.95	0.18
Overall average	2.00	0.10	0.36	0.46	0.09

Figure C.48 The total pipeline CPI and the contributions of the four major sources of stalls are shown. The major contributors are FP result stalls (both for branches and for FP inputs) and branch stalls, with loads and FP structural stalls adding less.

Pipelining topics covered so far



- RISC ISA
- 5 stage Integer Pipelined CPU
 - Performance
 - Hazards
- Stalls to improve RAW, WAR, WAW Data Hazards
- Reordering and NoOp in Software / Compilers
- Forwarding (or Bypassing) to Improve Data Hazards
- Interlock feature in Pipelined Hardware Architecture
- Control / Branch Hazards
 - Static / Dynamic / Flush / Restart
 - Branch Prediction
- Dealing with Exceptions / Interrupts in Pipelined Architecture
 - Status and Mask Registers
- Multicycle Floating point Pipelined RISC Architecture
 - Introduce Out of Order Execution
 - Introduce Scheduling of Instructions
- Case study of MIPS R4000 Pipeline
 - Instruction Memory and Data Memory with Cache support



Scoreboard in Dynamic Scheduling in RISC-V

RISC-V with Scoreboard

Multicycle EX Stage

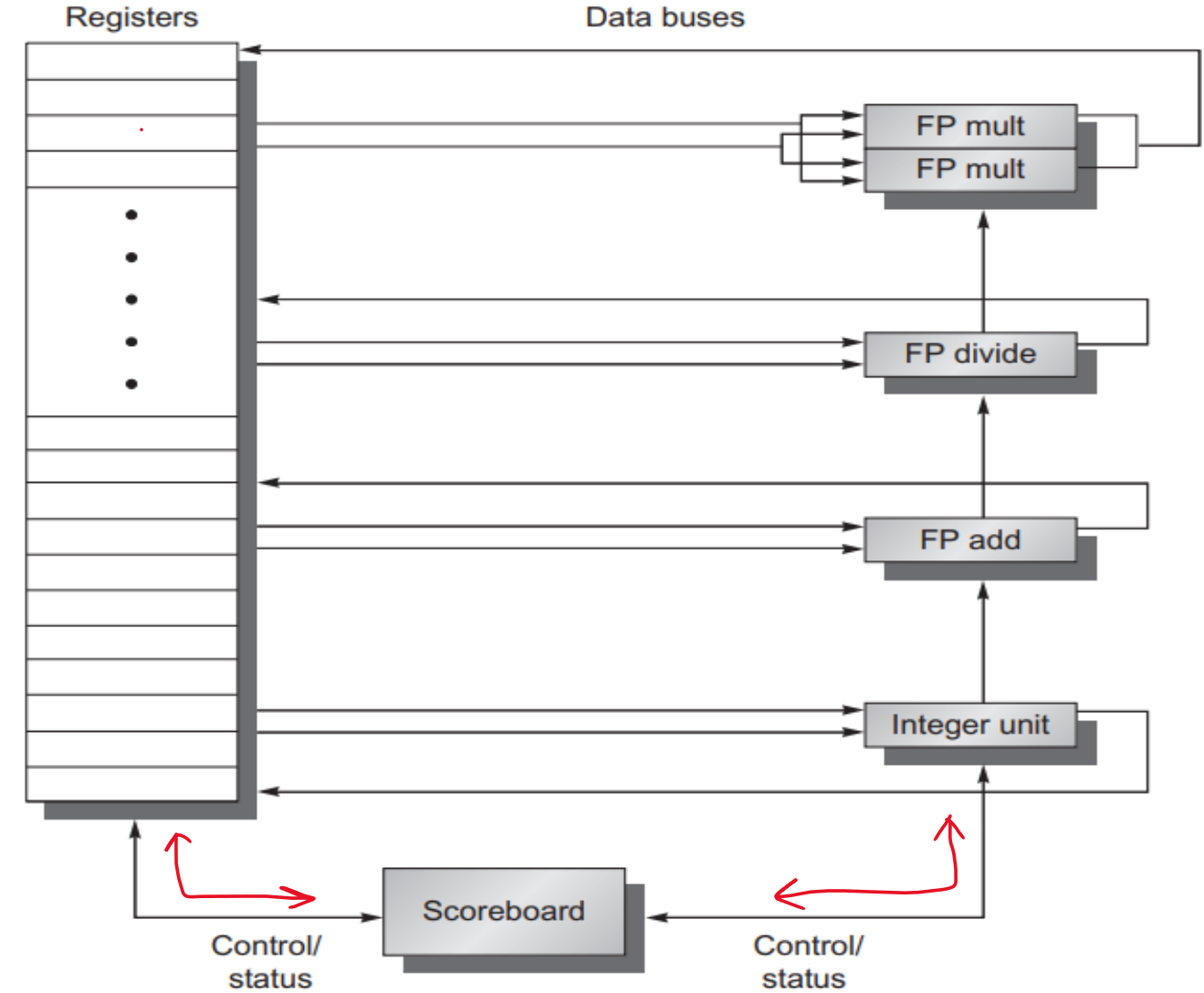


Figure C.49 The basic structure of a RISC V processor with a scoreboard. The scoreboard's function is to control instruction execution (vertical control lines). All of the data flow between the register file and the functional units over the buses (the horizontal lines, called *trunks* in the CDC 6600). There are two FP multipliers, an FP divider, an FP adder, and an integer unit. One set of buses (two inputs and one output) serves a group of functional units. We will explore scoreboarding and its extensions in more detail in Chapter 3.

- **Issue:** time at which instruction execute
- **Schedule:** order in which instruction execute
 - Related to issue, but the distinction is important
- **Scheduling:** re-arranging instruction enable rapid issue
 - Static: by compiler
 - Requires knowledge of pipeline and program dependences
 - **Pipeline scheduling:** the basics
 - Requires large scheduling scope full of independent insns
 - **Loop unrolling**, software pipelining: increase scope for loops
 - **Trace scheduling:** increase scope for non-loops

Microarchitects Perspective:

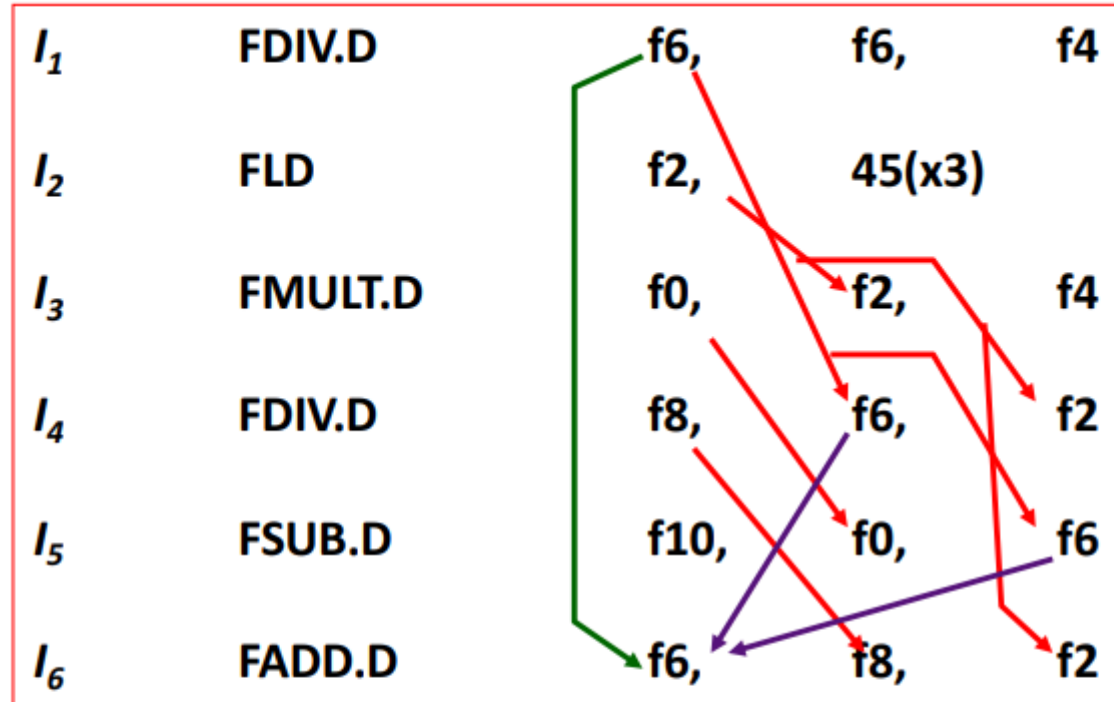
Anything software can do ... hardware can do better

- Rearrange order of instructions to reduce stalls while maintaining data flow
- Advantages:
 - Compiler doesn't need to have knowledge of microarchitecture
 - Handles cases where dependencies are unknown at compile time
- Disadvantage:
 - Substantial increase in hardware complexity
 - Complicates exceptions

Out-of-Order Execution

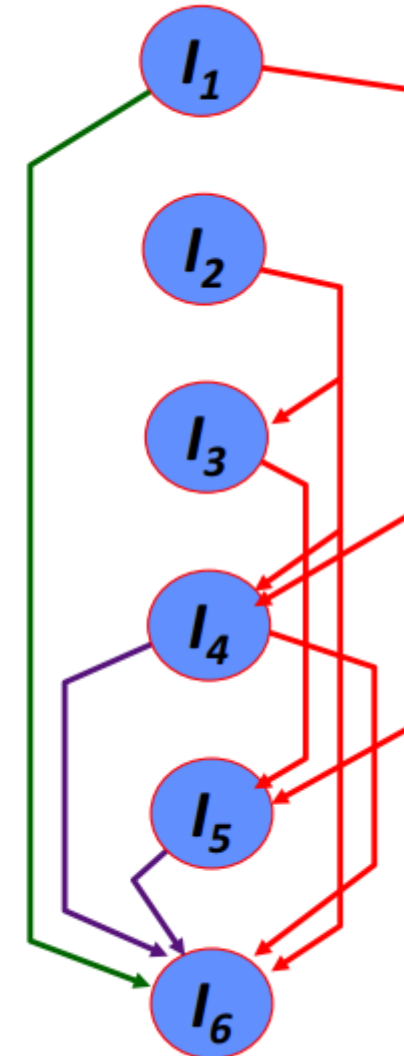
- Also called "Dynamic scheduling"
 - Done by the hardware on-the-fly during execution
- Looks at a "window" of instructions waiting to execute
 - Each cycle, picks the next ready instruction(s)
- Two steps to enable out-of-order execution:
 - • Step #1: Register renaming – to avoid "false" dependencies
 - • Step #2: Dynamically schedule – to enforce "true" dependencies
- Key to understanding out-of-order execution:
 - Build and execute a dependence graph (dataflow execution)
 - Data dependencies

Instruction Scheduling Example



Valid orderings:

in-order	I_1	I_2	I_3	I_4	I_5	I_6
out-of-order	I_2	I_1	I_3	I_4	I_5	I_6
out-of-order	I_1	I_2	I_3	I_5	I_4	I_6



- Dynamic scheduling implies:

- Out-of-order execution
- Out-of-order completion

- Example 1:

`fdiv.d f0,f2,f4`

`fadd.d f10,f0,f8`

`fsub.d f12,f8,f14`

- `fsub.d` is not dependent, can be issued before `fadd.d`

Dynamic Scheduling With a Scoreboard

1 → In a dynamically scheduled pipeline, all instructions pass through the issue stage in order (in-order issue); however, they can be stalled or bypass each other in the second stage (read operands) and thus enter execution out of order. Scoreboarding is a technique for allowing instructions to execute out of order when there are sufficient resources and no data dependences; it is named after the CDC 6600 scoreboard, which developed this capability.

2 ↗

Goal of Scoreboard technique



The goal of a scoreboard is to maintain an execution rate of one instruction per clock cycle (when there are no structural hazards) by executing an instruction as early as possible. Thus, when the next instruction to execute is stalled, other instructions can be issued and executed if they do not depend on any active or stalled instruction. The scoreboard takes full responsibility for instruction issue and execution, including all hazard detection. Taking advantage of out-of-order execution requires multiple instructions to be in their EX stage simultaneously. This can be achieved with multiple functional units, with pipelined functional units, or with both. Because these two capabilities—pipelined functional units and multiple functional units—are essentially equivalent for the purposes of pipeline control, we will assume the processor has multiple functional units.

What happens in a Scoreboard



Every instruction goes through the scoreboard, where a record of the data dependences is constructed; this step corresponds to instruction issue and replaces part of the ID step in the RISC V pipeline. The scoreboard then determines when the instruction can read its operands and begin execution. If the scoreboard decides the instruction cannot execute immediately, it monitors every change in the hardware and decides when the instruction *can* execute. The scoreboard also controls when an instruction can write its result into the destination register. Thus, all hazard detection and resolution are centralized in the scoreboard. We will see a picture of the scoreboard later (Figure C.49 on page C.68), but first we need to understand the steps in the issue and execution segment of the pipeline.

Four step Execution in Scoreboard



- **Issue**

If a functional unit for instruction is free and no other active instruction has the same destination register, the scoreboard Issues the instruction to the functional unit and updates its internal data structure. Like **IF** stage.

- **Read Operands**

The scoreboard monitors the availability of the source operands. A source operand is available if the earlier issued instruction is going to write it. When the source operands are available, the scoreboard tells the functional unit to proceed to read the operands from the registers and begin Execution. This step works together with **Issue** to form ID.

- **Execution**

The functional unit begins execution upon receiving operands. When the result is ready, it notifies the scoreboard that it has completed the execution.

This step **replaces EX** step in RISC-V pipeline and takes multiple cycles.

- **Write Result**

Once the scoreboard is aware that the functional unit has completed the execution, the scoreboard checks for WAR hazards and stalls the completing instruction, if necessary.

This is like **WB** stage in RISC-V.

WAR Hazards in Out of Order Execution

Before we see how scoreboarding could be used in the RISC V pipeline, it is important to observe that WAR hazards, which did not exist in the RISC V floating-point or integer pipelines, may arise when instructions execute out of order. For example, consider the following code sequence:

```

{ fdiv.d    f0, f2, f4
  fadd.d    f10, f0, f8
  fsub.d    f8, f8, f14

```

Handwritten annotations: A red bracket groups the three instructions. A red arrow points from `f0` in the first instruction to `f0` in the second. Another red arrow points from `f8` in the second instruction to `f8` in the third. The label "WT" (Write-Through) is written near the `f8` in the third instruction.

There is an potential WAR hazard between the `fadd.d` and the `fsub.d`: If the pipeline executes the `fsub.d` before the `fadd.d`, it will violate yield incorrect execution. Likewise, the pipeline must avoid WAW hazards (e.g., as would occur if the destination of the `fsub.d` were `f10`). As we will see, both these hazards are avoided in a scoreboard by stalling the later instruction involved in the hazard.

At first glance, it might appear that the scoreboard will have difficulty separating RAW and WAR hazards.

Because the operands for an instruction are read only when both operands are available in the register file, **this scoreboard does not take advantage of forwarding. Instead, registers are only read when they are both available.** This is not as large a penalty as you might initially think. Unlike our simple pipeline of earlier, **instructions will write their result into the register file as soon as they complete execution (assuming no WAR hazards), rather than wait for a statically assigned write slot that may be several cycles away.** The effect reduces the pipeline latency and the benefits of forwarding. There is still one additional cycle of latency that arises because the write result and read operand stages cannot overlap. We would need additional buffering to eliminate this overhead.

Chap 3: Instruction Level Parallelism

Starting Chapter 3 of textbook

First-order Architecture Questions

- When deciding whether to add some feature:
 - Why should we do it?
 - Usually more parallelism (more IPC)
 - Why shouldn't we do it?
 - Complexity + Power/Energy
 - **Less obvious: Who should do it?**
 - **Hardware:**
 - Access to dynamic information + fine-grain control
 - Less annoying to add interfaces...
 - **Compiler+ISA:**
 - Access to high-level information + off critical path

What Impacts CPI? Pipelines and Stalls



In this section, we discuss features of both programs and processors that limit the amount of parallelism that can be exploited among instructions, as well as the critical mapping between program structure and hardware structure, which is key to understanding whether a program property will actually limit performance and under what circumstances.

The value of the **CPI** (cycles per instruction) for a pipelined processor is the **sum of the base CPI and all contributions from stalls:**

$$\text{Pipeline CPI} = \text{Ideal pipeline CPI} + \text{Structural stalls} + \text{Data hazard stalls} + \text{Control stalls}$$

Instruction-Level Parallelism

- When exploiting instruction-level parallelism, goal is to maximize CPI

- Pipeline CPI =

- Ideal pipeline CPI +
- Structural stalls +
- Data hazard stalls +
- Control stalls

- Parallelism with basic block is limited

- Typical size of basic block = 3 – 6 instructions
- Must optimize across branches

Instr inside loop | Branch



- Pipelining become universal technique in 1985
 - Overlaps execution of instructions
 - Exploits “Instruction Level Parallelism”
- Beyond this, there are two main approaches:
 - Hardware-based dynamic approaches
 - Used in server and desktop processors
 - Not used as extensively in IoT, Low Power, processors
 - Compiler-based static approaches
 - Not as successful outside of scientific applications

What Is Instruction-Level Parallelism?

All the techniques in this chapter exploit parallelism among instructions. The amount of parallelism available within a *basic block*—a straight-line code sequence with no branches in except to the entry and no branches out except at the exit—is quite small. For typical RISC programs, the average dynamic branch frequency is often between 15% and 25%, meaning that between three and six instructions execute between a pair of branches. Because these instructions are likely to depend upon one another, the amount of overlap we can exploit within a basic block is likely to be less than the average basic block size. To obtain substantial performance enhancements, we must exploit ILP across multiple basic blocks.

- **Pipeline scheduling**

- Separate dependent instruction from the source instruction by the pipeline latency of the source instruction

- **Example:**

```
for (i=999; i>=0; i=i-1)
    x[i] = x[i] + s;
```

Compiler based

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

Pipeline Stalls

Loop: fld f0,0(x1)
 stall
 fadd.d f4,f0,f2
 stall
 stall
 fsd f4,0(x1)
 addi x1,x1,-8
 bne x1,x2,Loop

Loop: fld f0,0(x1)
 addi x1,x1,-8
 fadd.d f4,f0,f2
 stall
 stall
 fsd f4,0(x1)
 bne x1,x2,Loop

Compiler Techniques

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

- Dependencies are a property of programs
- Pipeline organization determines if dependence is detected and if it causes a stall
- Data dependence conveys:
 - Possibility of a hazard
 - Order in which results must be calculated
 - Upper bound on exploitable instruction level parallelism
- Dependencies that flow through memory locations are difficult to detect

Data Dependences and Hazards

Determining how one instruction depends on another is critical to determining how much parallelism exists in a program and how that parallelism can be exploited. In particular, to exploit instruction-level parallelism, we must determine which instructions can be executed in parallel. If two instructions are parallel, they can execute simultaneously in a pipeline of arbitrary depth without causing any stalls, assuming the pipeline has sufficient resources (and thus no structural hazards exist). If two instructions are dependent, they are not parallel and must be executed in order, although they may often be partially overlapped. The key in both cases is to determine whether an instruction is dependent on another instruction.

Dependence - value stored in Registers or in Memory



A data value may flow between instructions either through registers or through memory locations. When the data flow occurs through a register, detecting the dependence is straightforward because the register names are fixed in the instructions, although it gets more complicated when branches intervene and correctness concerns force a compiler or hardware to be conservative.

Dependences that flow through memory locations are more difficult to detect because two addresses may refer to the same location but look different: For example, $100(x4)$ and $20(x6)$ may be identical memory addresses. In addition, the effective address of a load or store may change from one execution of the instruction to another (so that $20(x4)$ and $20(x4)$ may be different), further complicating the detection of a dependence.

In this chapter, we examine hardware for detecting data dependences that involve memory locations, but we will see that these techniques also have limitations. The compiler techniques for detecting such dependences are critical in uncovering loop-level parallelism.

Overcome Dependence



Because a data dependence can limit the amount of instruction-level parallelism we can exploit, a major focus of this chapter is overcoming these limitations. A dependence can be overcome in two different ways: (1) maintaining the dependence but avoiding a hazard, and (2) eliminating a dependence by transforming the code. Scheduling the code is the primary method used to avoid a hazard without altering a dependence, and such scheduling can be done both by the compiler and by the hardware.

Dependence Types



- Data Dependence
- Name Dependence
- Control Dependence

- Data Hazards
 - Read after write (RAW)
 - Write after write (WAW)
 - Write after read (WAR)
- Control Dependence
 - Ordering of instruction i with respect to a branch instruction
 - Instruction control dependent on a branch cannot be moved before the branch so that its execution is no longer controlled by the branch
 - An instruction not control dependent on a branch cannot be moved after the branch so that its execution is controlled by the branch

- Example 1:

```
add x1,x2,x3  
beq x4,x0,L  
sub x1,x1,x6
```

```
L: ...  
or x7,x1,x8
```

- or instruction dependent on add and sub

- Example 2:

```
add x1,x2,x3  
beq x12,x0,skip  
sub x4,x5,x6  
add x5,x4,x9
```

```
skip:  
or x7,x8,x9
```

- Assume x4 isn't used after skip
 - Possible to move sub before the branch

Dependence types – code examples



- RAW (Read After Write) = "true dependence"

mul r0 * r1 -> **r2**

...

add **r2** + r3 -> r4

- WAW (Write After Write) = "output dependence"

mul r0 * r1 -> **r2**

...

add r1 + r3 -> **r2**

- WAR (Write After Read) = "anti-dependence"

mul r0 * **r1** -> r2

...

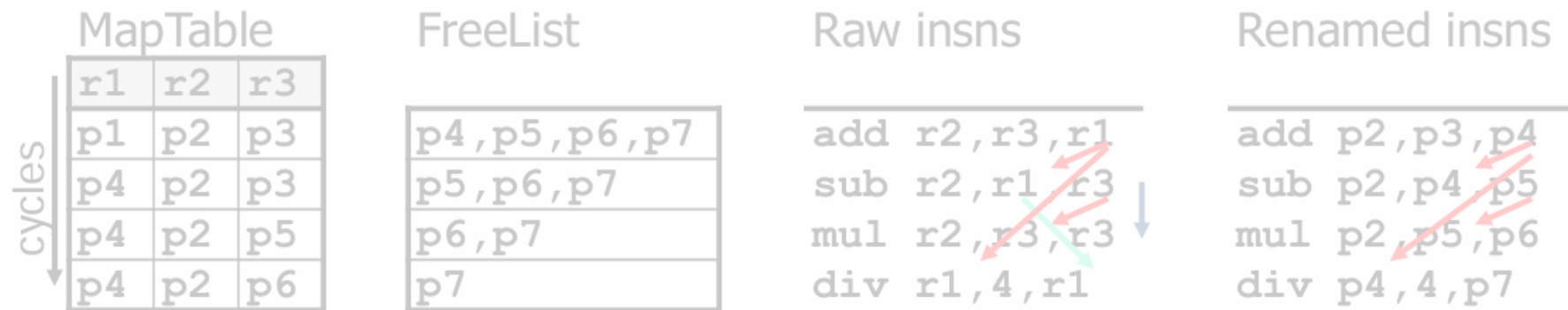
add r3 + r4 -> **r1**

- WAW & WAR are "false", Can be totally eliminated by "renaming"

Register Renaming

Register Renaming

- To eliminate WAW and WAR hazards
- Example
 - Names: `r1, r2, r3` (logical registers)
 - Locations: `p1, p2, p3, p4, p5, p6, p7` (physical registers)
 - Original mapping: `r1 → p1, r2 → p2, r3 → p3`, `p4–p7` are "free"



- Renaming
 - + Removes **WAW** and **WAR** dependences
 - + Leaves **RAW** intact!

Name Dependence and Output Dependence

- An **Antidependence** between instruction i and instruction j occurs when instruction j writes a register or memory that instruction i reads.
 - The original ordering must be preserved to ensure that i reads the correct value. In example, there is antidependence between fsd and addi on register x1.
- An **Output dependence** occurs when instruction i and instruction j write the same register or memory location.
 - The ordering between the instruction must be preserved to ensure that value finally written corresponds to instruction j.

• Both Antidependence and Output Dependence are **Name Dependences**, as opposed to True Dependences, as no value is being transmitted between the two instructions.

```

Loop: fld    f0,0(x1)    //f0=array element
      fadd.d f4,f0,f2    //add scalar in f2
      fsd    f4,0(x1)    //store result
      addi   x1,x1,-8     //decrement pointer 8 bytes
      bne    x1,x2,Loop  //branch x1≠x2
    
```

The data dependences in this code sequence involve both floating-point data:

```

Loop: fld    f0,0(x1)    //f0=array element
      fadd.d f4,f0,f2    //add scalar in f2
      fsd    f4,0(x1)    //store result
    
```

and integer data:

```

      addi   x1,x1,-8     //decrement pointer
                          //8 bytes (per DW)
      bne    x1,x2,Loop  //branch x1≠x2
    
```

Dependence Chain

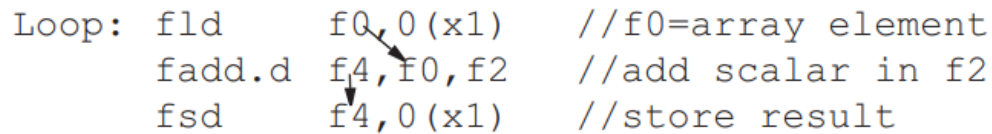
This dependence chain can be as long as the entire program. Note that a dependence within a single instruction (such as `add x1, x1, x1`) is not considered a dependence.

For example, consider the following RISC-V code sequence that increments a vector of values in memory (starting at `0(x1)` ending with the last element at `0(x2)`) by a scalar in register `f2`.

```
Loop: fld      f0,0(x1)    //f0=array element
      fadd.d   f4,f0,f2    //add scalar in f2
      fsd      f4,0(x1)    //store result
      addi     x1,x1,-8     //decrement pointer 8 bytes
      bne      x1,x2,Loop  //branch x1≠x2
```

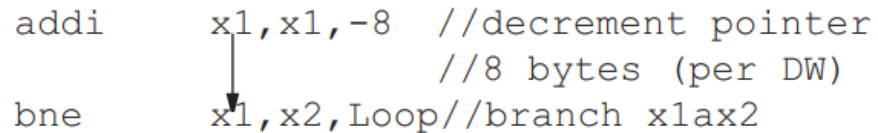
The data dependences in this code sequence involve both floating-point data:

```
Loop: fld      f0,0(x1)    //f0=array element
      fadd.d   f4,f0,f2    //add scalar in f2
      fsd      f4,0(x1)    //store result
```



and integer data:

```
addi     x1,x1,-8     //decrement pointer
                          //8 bytes (per DW)
bne      x1,x2,Loop  //branch x1≠x2
```



In both of the preceding dependent sequences, as shown by the arrows, each instruction depends on the previous one. The arrows here and in following examples show the order that must be preserved for correct execution. The arrow points from an instruction that must precede the instruction that the arrowhead points to.

More on Data Dependences

Dependences are a property of *programs*. Whether a given dependence results in an *actual hazard being detected* and *whether that hazard actually causes a stall* are properties of the *pipeline organization*. This difference is critical to understanding how instruction-level parallelism can be exploited.

A data dependence conveys three things: (1) *the possibility of a hazard*, (2) *the order in which results must be calculated*, and (3) *an upper bound on how much parallelism can possibly be exploited*. Such limits are explored in a pitfall on [page 262](#) and in Appendix H in more detail.

Because a data dependence can limit the amount of instruction-level parallelism we can exploit, a major focus of this chapter is overcoming these limitations. A dependence can be overcome in two different ways: (1) *maintaining the dependence but avoiding a hazard*, and (2) *eliminating a dependence by transforming the code*. Scheduling the code is the primary method used to avoid a hazard without altering a dependence, and such scheduling can be done both by the compiler and by the hardware.

A data value may flow between instructions either through registers or through memory locations. When the data flow occurs through a register, detecting the dependence is straightforward because the register names are fixed in the instructions, although it gets more complicated when branches intervene and correctness concerns force a compiler or hardware to be conservative.

Loop Unrolling ILP

Basic Compiler Techniques for Exposing ILP

This section examines the use of simple **compiler technology to enhance a processor's ability to exploit ILP. These techniques are crucial for processors that use static issue or static scheduling.** Armed with this compiler technology, we will shortly examine the design and performance of processors using static issuing. Appendix H will investigate more sophisticated compiler and associated hardware schemes designed to enable a processor to exploit more instruction-level parallelism.

Basic Pipeline Scheduling and Loop Unrolling

To keep a pipeline full, parallelism among instructions must be exploited by finding sequences of unrelated instructions that can be overlapped in the pipeline. To **avoid a pipeline stall, the execution of a dependent instruction must be separated from the source instruction by a distance in clock cycles equal to the pipeline latency of that source instruction.** A compiler's ability to perform this scheduling depends both on the amount of ILP available in the program and on the latencies of the functional units in the pipeline. **Figure 3.2** shows the FP unit latencies we

Summary of the Loop Unrolling and Scheduling

Throughout this chapter and Appendix H, we will look at a variety of hardware and software techniques that allow us to take advantage of instruction-level parallelism to fully utilize the potential of the functional units in a processor. The key to most of these techniques is to know when and how the ordering among instructions may be changed. In our example, we made many such changes, which to us, as human beings, were obviously allowable. In practice, this process must be performed in a methodical fashion either by a compiler or by hardware. To obtain the final unrolled code, we had to make the following decisions and transformations:

- Determine that unrolling the loop would be useful by finding that the loop iterations were independent, except for the loop maintenance code.
- Use different registers to avoid unnecessary constraints that would be forced by using the same registers for different computations (e.g., name dependences).
- Eliminate the extra test and branch instructions and adjust the loop termination and iteration code.
- Determine that the loads and stores in the unrolled loop can be interchanged by observing that the loads and stores from different iterations are independent. This transformation requires analyzing the memory addresses and finding that they do not refer to the same address.
- Schedule the code, preserving any dependences needed to yield the same result as the original code.

Assembly Code of a Loop

In this section, we look at how the compiler can increase the amount of available ILP by transforming loops. This example serves both to illustrate an important technique as well as to motivate the more powerful program transformations described in Appendix H. We will rely on the following code segment, which adds a scalar to a vector:

```
for (i=999; i>=0; i=i-1)
    x[i] = x[i] + s;
```

The first step is to translate the preceding segment to RISC-V assembly language. In the following code segment, `x1` is initially the address of the element in the array with the highest address, and `f2` contains the scalar value `s`. Register `x2` is precomputed so that `Regs[x2]+8` is the address of the last element to operate on.

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

Figure 3.2 Latencies of FP operations used in this chapter. The last column is the number of intervening clock cycles needed to avoid a stall. These numbers are similar to the average latencies we would see on an FP unit. The latency of a floating-point load to a store is 0 because the result of the load can be bypassed without stalling the store. We will continue to assume an integer load latency of 1 and an integer ALU operation latency of 0 (which includes ALU operation to branch).

The straightforward RISC-V code, not scheduled for the pipeline, looks like this:

```
Loop: fld    f0,0(x1)      //f0=array element
      fadd.d  f4,f0,f2     //add scalar in f2
      fsd     f4,0(x1)     //store result
      addi    x1,x1,-8     //decrement pointer
                               //8 bytes (per DW)
      bne     x1,x2,Loop   //branch x1≠x2
```

Let's start by seeing how well this loop will run when it is scheduled on a simple pipeline for RISC-V with the latencies in [Figure 3.2](#).

Scheduling of Assembly Code

The first step is to translate the preceding segment to RISC-V assembly language. In the following code segment, `x1` is initially the address of the element in the array with the highest address, and `f2` contains the scalar value `s`. Register `x2` is precomputed so that `Regs[x2]+8` is the address of the last element to operate on.

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

Figure 3.2 Latencies of FP operations used in this chapter. The last column is the number of intervening clock cycles needed to avoid a stall. These numbers are similar to the average latencies we would see on an FP unit. The latency of a floating-point load to a store is 0 because the result of the load can be bypassed without stalling the store. We will continue to assume an integer load latency of 1 and an integer ALU operation latency of 0 (which includes ALU operation to branch).

Example Show how the loop would look on RISC-V, both scheduled and unscheduled, including any stalls or idle clock cycles. Schedule for delays from floating-point operations.

Answer Without any scheduling, the loop will execute as follows, taking nine cycles:

		Clock cycle issued
Loop:	<code>fld f0,0(x1)</code>	1
	<i>stall</i>	2
	<code>fadd.d f4,f0,f2</code>	3
	<i>stall</i>	4
	<i>stall</i>	5
	<code>fsd f4,0(x1)</code>	6
	<code>addi x1,x1,-8</code>	7
	<code>bne x1,x2,Loop</code>	8

We can schedule the loop to obtain only two stalls and reduce the time to seven cycles:

Loop:	<code>fld f0,0(x1)</code>
	<code>addi x1,x1,-8</code>
	<code>fadd.d f4,f0,f2</code>
	<i>stall</i>
	<i>stall</i>
	<code>fsd f4,8(x1)</code>
	<code>bne x1,x2,Loop</code>

The stalls after `fadd.d` are for use by the `fsd`, and repositioning the `addi` prevents the stall after the `fld`.

Loop Unrolled Assembly Code

Example Show our loop unrolled so that there are four copies of the loop body, assuming $x1 - x2$ (that is, the size of the array) is initially a multiple of 32, which means that the number of loop iterations is a multiple of 4. Eliminate any obviously redundant computations and do not reuse any of the registers.

Answer Here is the result after merging the `addi` instructions and dropping the unnecessary `bne` operations that are duplicated during unrolling. Note that $x2$ must now be set so that $\text{Regs}[x2]+32$ is the starting address of the last four elements.

The straightforward RISC-V code, not scheduled for the pipeline, looks like this:

```
Loop: fld      f0,0(x1)      //f0=array element
      fadd.d   f4,f0,f2      //add scalar in f2
      fsd      f4,0(x1)      //store result
      addi     x1,x1,-8       //decrement pointer
                               //8 bytes (per DW)
      bne      x1,x2,Loop     //branch x1≠x2
```

Let's start by seeing how well this loop will run when it is scheduled on a simple pipeline for RISC-V with the latencies in [Figure 3.2](#).

```
Loop: fld      f0,0(x1)
      fadd.d   f4,f0,f2
      fsd      f4,0(x1)      //drop addi & bne
      fld      f6,-8(x1)
      fadd.d   f8,f6,f2
      fsd      f8,-8(x1)     //drop addi & bne
      fld      f0,-16(x1)
      fadd.d   f12,f0,f2
      fsd      f12,-16(x1)   //drop addi & bne
      fld      f14,-24(x1)
      fadd.d   f16,f14,f2
      fsd      f16,-24(x1)
      addi     x1,x1,-32
      bne      x1,x2,Loop
```

We have eliminated three branches and three decrements of $x1$. The addresses on the loads and stores have been compensated to allow the `addi` instructions on $x1$ to be merged. This optimization may seem trivial, but it is not; it requires symbolic substitution and simplification. Symbolic substitution and simplification will rearrange expressions so as to allow constants to be collapsed, allowing an expression such as $((i+1)+1)$ to be rewritten as $(i+(1+1))$ and then simplified to $(i+2)$.

Incorporating Scheduling in Unrolled Code

Example Show the unrolled loop in the previous example after it has been scheduled for the pipeline with the latencies in Figure 3.2.

```

Loop: fld    f0,0(x1)
      fadd.d  f4,f0,f2
      fsd     f4,0(x1)      //drop addi & bne
      fld     f6,-8(x1)
      -----
      fadd.d  f8,f6,f2
      fsd     f8,-8(x1)     //drop addi & bne
      fld     f0,-16(x1)
      fadd.d  f12,f0,f2
      fsd     f12,-16(x1)  //drop addi & bne
      -----
      fld     f14,-24(x1)
      fadd.d  f16,f14,f2
      fsd     f16,-24(x1)
      addi    x1,x1,-32
      bne     x1,x2,Loop
  
```

Answer

```

Loop: fld    f0,0(x1)
      fld     f6,-8(x1)
      fld     f0,-16(x1)
      fld     f14,-24(x1)
      fadd.d  f4,f0,f2
      fadd.d  f8,f6,f2
      fadd.d  f12,f0,f2
      fadd.d  f16,f14,f2
      fsd     f4,0(x1)
      fsd     f8,-8(x1)
      fsd     f12,16(x1)
      fsd     f16,8(x1)
      addi    x1,x1,-32
      bne     x1,x2,Loop
  
```

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

Figure 3.2 Latencies of FP operations used in this chapter. The last column is the number of intervening clock cycles needed to avoid a stall. These numbers are similar to the average latencies we would see on an FP unit. The latency of a floating-point load to a store is 0 because the result of the load can be bypassed without stalling the store. We will continue to assume an integer load latency of 1 and an integer ALU operation latency of 0 (which includes ALU operation to branch).

The execution time of the unrolled loop has dropped to a total of 14 clock cycles, or 3.5 clock cycles per element, compared with 8 cycles per element before any unrolling or scheduling and 6.5 cycles when unrolled but not scheduled.

- Loop unrolling
 - Unroll by a factor of 4 (assume # elements is divisible by 4)
 - Eliminate unnecessary instructions

Compiler Techniques

```
Loop:      fld f0,0(x1)
           fadd.d f4,f0,f2
           fsd f4,0(x1) //drop addi & bne
           fld f6,-8(x1)
           fadd.d f8,f6,f2
           fsd f8,-8(x1) //drop addi & bne
           fld f0,-16(x1)
           fadd.d f12,f0,f2
           fsd f12,-16(x1) //drop addi & bne
           fld f14,-24(x1)
           fadd.d f16,f14,f2
           fsd f16,-24(x1)
           addi x1,x1,-32
           bne x1,x2,Loop
```

■ note: number of
live registers vs.
original loop

- Pipeline schedule the unrolled loop:

```
Loop:   fld f0,0(x1)
        fld f6,-8(x1)
        fld f8,-16(x1)
        fld f14,-24(x1)
        fadd.d f4,f0,f2
        fadd.d f8,f6,f2
        fadd.d f12,f0,f2
        fadd.d f16,f14,f2
        fsd f4,0(x1)
        fsd f8,-8(x1)
        fsd f12,-16(x1)
        fsd f16,-24(x1)
        addi x1,x1,-32
        bne x1,x2,Loop
```

Compiler Techniques

- 14 cycles
- 3.5 cycles per element